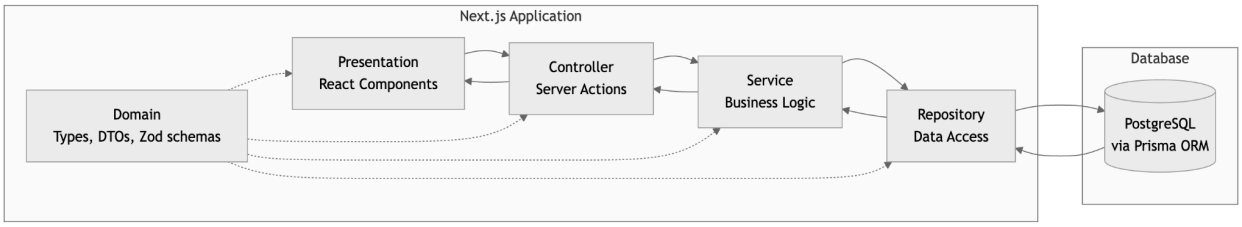
Gym Tracker Pro

Design Specification

Architectural Overview

GymTracker Pro follows a classic MVC pattern layered inside Next.js App Router. All server-side logic lives under /lib, split into four sub-layers.

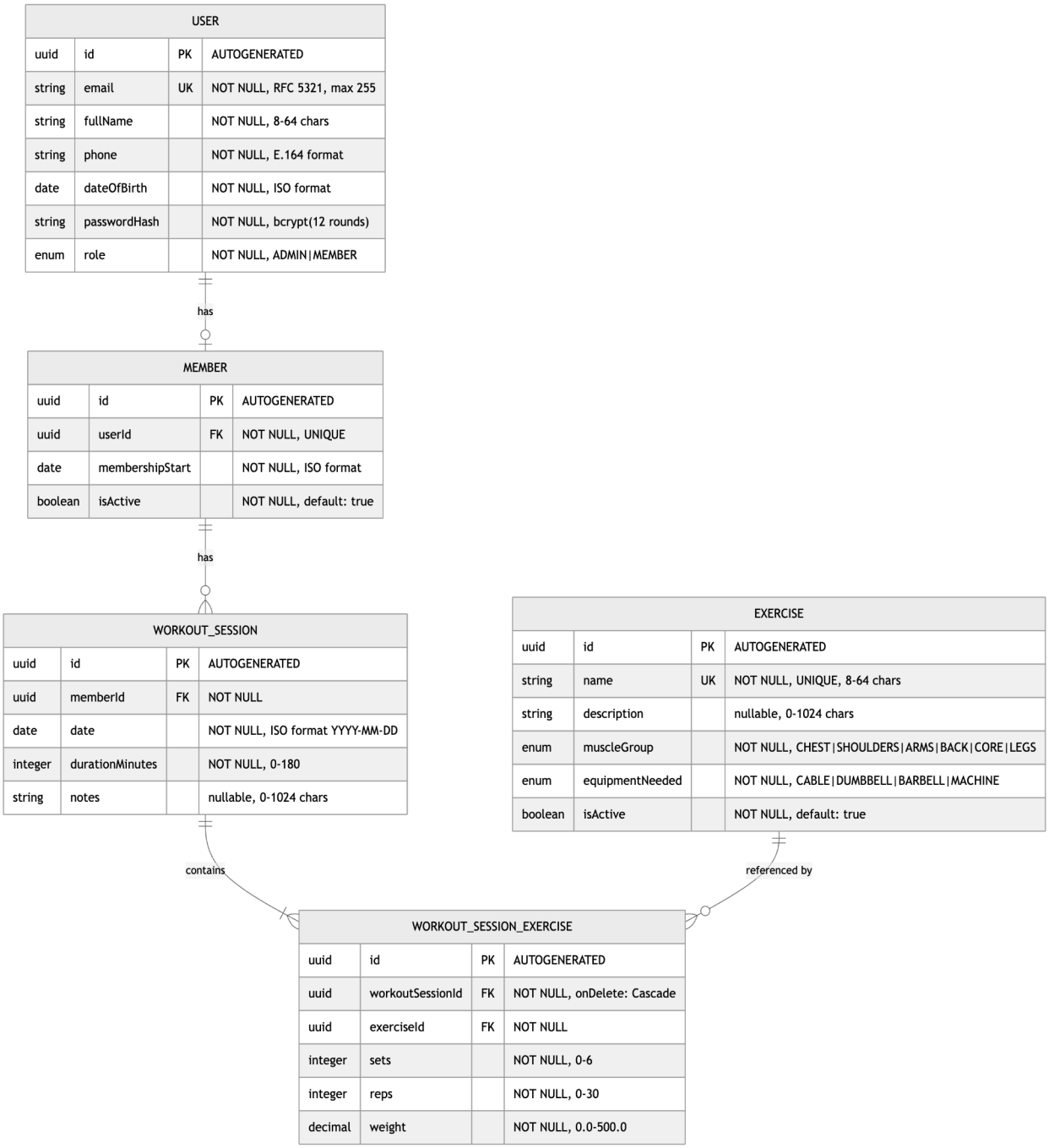


Layer Communication Rules

Layer	Imports from	Does NOT import from
Presentation Layer	Controller Layer, Domain Layer	Service Layer, Repository Layer, Database via Prisma ORM
Controller Layer	Service Layer, Domain Layer	Presentation Layer, Repository Layer, Database via Prisma ORM
Service Layer	Repository Layer, Domain Layer	Presentation Layer, Controller Layer, Database via Prisma ORM
Repository Layer	Domain Layer, Database via Prisma ORM	Presentation Layer, Controller Layer, Service Layer
Domain Layer	Database via Prisma ORM	Presentation Layer, Controller Layer, Service Layer, Repository Layer

Entity & Data Model

Entity Relationship Diagram



Data Integrity Invariants

- A WorkoutSession cannot exist without a valid memberId.
- A SessionExercise cannot exist without a valid workoutSessionId and exerciseId.
- Deleting a WorkoutSession cascades to all its SessionExercise rows.
- Archiving an Exercise does NOT cascade - existing SessionExercise rows retain the reference for historical accuracy.
- Deactivating a Member does NOT delete their sessions.

Module Responsibilities

Domain Layer

File	Responsibility
user.ts	Exports Prisma-generated **User** type, **CreateUser** type, **UpdateUser** type, and **Role** enum (ADMIN, MEMBER).
user-schema.ts	Exports zod validation schemas: **createUserSchema**, **updateUserSchema**, **loginUserSchema**.
member.ts	Exports Prisma-generated **Member** type, **CreateMember** type, **UpdateMember** type.
member-schema.ts	Exports zod validation schemas: **createMemberSchema**, **updateMemberSchema**.
exercise.ts	Exports Prisma-generated **Exercise** type, **CreateExercise** type, **UpdateExercise** type, **MuscleGroup** enum (CHEST, SHOULDERS, ARMS, BACK, CORE, LEGS), **Equipment** enum (CABLE, DUMBBELL, BARBELL, MACHINE).
exercise-schema.ts	Exports zod validation schemas: **createExerciseSchema**, **updateExerciseSchema**.
workout-session.ts	Exports Prisma-generated **WorkoutSession** type, **CreateWorkoutSession** type, **UpdateWorkoutSession** type.
workout-session-schema.ts	Exports zod validation schemas: **createWorkoutSessionSchema**, **updateWorkoutSessionSchema**.
workout-session-exercise.ts	Exports Prisma-generated **WorkoutSessionExercise** type, **CreateWorkoutSessionExercise** type, **UpdateWorkoutSessionExercise** type.
workout-session-exercise-schema.ts	Exports zod validation schemas: **createWorkoutSessionExerciseSchema**, **updateWorkoutSessionExerciseSchema**.
errors.ts	Exports custom error classes: **AppError**, **NotFoundError**, **ConflictError**, **AuthorizationError**, **SessionRequiresExercisesError**, **TransactionError**. Thrown by services and caught by controllers.
report.ts	Exports **Report** type (totalSessions, totalVolume, averageSessionDuration, exerciseBreakdown[],

	sessionDetails[]), **ExerciseStats** type (exerciseId, exerciseName, totalSets, maxWeightKg), **SessionDetail** type (sessionId, date, durationMinutes, exercises[]), and **SessionExerciseDetail** type (exerciseName, sets, reps, weight).
session.ts	Exports **Session** type (userId, email, fullName, role, memberId?). Shape of user object stored in server session cookie.

Repository Layer

File	Responsibility
user-repository.ts	Exports the following functions: ● **create(userData: CreateUser): Promise<User>** - Create user with hashed password ● **findById(id: string): Promise<User | null>** - Get user by UUID ● **findByEmail(email: string): Promise<User | null>** - Get user by email (login) ● **update(id: string, userData: UpdateUser): Promise<User>** - Update user fields ● **delete(id: string): Promise<void>** - Hard delete user
member-repository.ts	Exports the following functions: ● **create(memberData: CreateMember): Promise<Member>** - Create member linked to user ● **findById(id: string): Promise<Member | null>** - Get member by UUID ● **findByEmail(email: string): Promise<Member | null>** - Check email uniqueness ● **findByUserId(userId: string): Promise<Member | null>** - Get member by user FK ● **findPaginated(page: number, pageSize?: number, searchQuery?: string): Promise<Member[]}>** - List members (10/page), searchable by fullName, sorted ascending ● **update(id: string, memberData: UpdateMember): Promise<Member>** - Update membershipStart, isActive ● **updateWithUserFields(id: string, memberData: UpdateMember, userData: UpdateUser): Promise<Member>** -atomic update Member and linked User fields in single transaction ● **suspend(id: string): Promise<Member>** - Set isActive = false ● **unsuspend(id: string): Promise<Member>** - Set isActive = true

	• **delete(id: string): Promise<void>** - atomic hard delete member and cascade user in single transaction
exercise-repository.ts	Exports the following functions: • **create(exerciseData: CreateExercise): Promise<Exercise>** - Create exercise • **findById(id: string): Promise<Exercise | null>** - Get exercise by UUID • **findByName(name: string): Promise<Exercise | null>** - Check exercise name uniqueness • **findPaginated(page: number, pageSize?: number, searchQuery?: string, includeInactive?: boolean): Promise<Exercise[]>** - List exercises (10/page), searchable by name, sorted ascending. Default excludes archived • **findActive(): Promise<Exercise[]>** - List all active exercises • **update(id: string, exerciseData: UpdateExercise): Promise<Exercise>** - Update name, description, muscleGroup, equipmentNeeded • **archive(id: string): Promise<Exercise>** - Set isActive = false (soft delete) • **unarchive(id: string): Promise<Exercise>** - Set isActive = true • **delete(id: string): Promise<void>** - Hard delete exercise
workout-session-repo sitory.ts	Exports the following functions: • **create(sessionData: CreateWorkoutSession, sessionExercises: CreateWorkoutSessionExercise[]): Promise<WorkoutSession>** - atomic create WorkoutSession and all SessionExercise records in single database transaction. Min 1 exercise required (WSM-04) • **findById(id: string): Promise<WorkoutSession | null>** - Get session with nested exercises • **findByIdWithDetails(id: string): Promise<WorkoutSession | null>** - Get session with exercise details (joins to Exercise table) • **findByMemberPaginated(memberId: string, page: number, pageSize?: number): Promise<WorkoutSession[]>** - List member's sessions (10/page), sorted by date descending, includes exercises • **update(id: string, sessionData: UpdateWorkoutSession): Promise<WorkoutSession>** - Update date, durationMinutes, notes (does NOT modify exercises - use workout-session-exercise-repository) • **findByMemberAndDateRange(memberId: string, startDate: Date, endDate: Date): Promise<WorkoutSession[]>** - Get sessions in date range, sorted descending • **countByMemberAndDateRange(memberId: string,**

	startDate: Date, endDate: Date): Promise<number> - Count sessions in range (for reports) ● **delete(id: string): Promise<void>** - Hard delete session and cascade exercises via workout-session-exercise-repository.deleteByWorkoutSessionId()
workout-session-exercise-repository.ts	Exports the following functions: ● **create(exerciseData: CreateWorkoutSessionExercise): Promise<WorkoutSessionExercise>** - Create session exercise (called by workout-session-repository.create() atomically, or individually via addSessionExercise) ● **findById(id: string): Promise<WorkoutSessionExercise | null>** - Get session exercise by ID ● **findBySessionId(sessionId: string): Promise<WorkoutSessionExercise[]>** - Get all exercises for a session (with exercise details joined) ● **countBySessionId(sessionId: string): Promise<number>** - Count exercises in a session ● **update(id: string, exerciseData: UpdateWorkoutSessionExercise): Promise<WorkoutSessionExercise>** - Update sets, reps, weight ● **delete(id: string): Promise<void>** - Delete session exercise ● **deleteByWorkoutSessionId(sessionId: string): Promise<void>** - Delete all exercises for a session (cascade on session deletion)
report-repository.ts	Exports the following functions: ● **getProgressReportData(memberId: string, startDate: Date, endDate: Date): Promise<Report>** - Server-side aggregation returning totalSessions, totalVolume (sum of sets×reps×weight), averageSessionDuration, exerciseBreakdown[], sessionDetails[] ● **getTotalVolume(memberId: string, startDate: Date, endDate: Date): Promise<number>** - Sum of (sets × reps × weight) ● **getExerciseBreakdown(memberId: string, startDate: Date, endDate: Date): Promise<ExerciseStats[]>** - Per-exercise: totalSets, maxWeightKg ● **getSessionSummary(memberId: string, startDate: Date, endDate: Date): Promise<SessionDetail[]>** - Chronological sessions with nested exercises

Service Layer

File	Responsibility

auth-service.ts	Exports the following functions: ● **login(email: string, password: string): Promise<User>** - Validate credentials, throw AuthorizationError if invalid ● **hashPassword(password: string): Promise<string>** - Hash plain password using bcrypt (12 rounds) ● **verifyPassword(plainPassword: string, hash: string): Promise<boolean>** - Compare plain password with hash ● **generateTempPassword(): string** - Generate 8-64 char password with uppercase, number, special char
user-service.ts	Exports the following functions: ● **createUser(userData: CreateUser): Promise<User>** - Validate input via schema, delegate to repository ● **getUserById(id: string): Promise<User>** - Fetch user, throw NotFoundError if not found ● **getUserByEmail(email: string): Promise<User>** - Fetch user by email, throw NotFoundError if not found ● **updateUser(id: string, userData: UpdateUser): Promise<User>** - Validate input via schema, delegate to repository ● **deleteUser(id: string): Promise<void>** - Delete user, throw NotFoundError if not found
member-service.ts	Exports the following functions: ● **createMember(fullName: string, email: string, phone: string, dateOfBirth: Date, membershipStart: Date): Promise<Member>** - Create user and member atomically. Hash temp password. Return member with temp password. Throw ConflictError if email exists ● **listMembers(page: number, searchQuery?: string): Promise<Member[]>** - Fetch paginated members from repository ● **getMemberById(id: string): Promise<Member>** - Fetch member, throw NotFoundError if not found ● **getMemberByUserId(userId: string): Promise<Member>** - Fetch member by user FK, throw NotFoundError if not found ● **updateMember(id: string, fullName?: string, email?: string, phone?: string, dateOfBirth?: Date, membershipStart?: Date): Promise<Member>** - Validate inputs via schemas, call repository.updateWithUserFields() atomically. Throw ConflictError if email duplicate. Throw NotFoundError if member not found ● **suspendMember(id: string): Promise<Member>** - Set isActive = false. Throw NotFoundError if not found ● **unsuspendMember(id: string): Promise<Member>** - Set isActive = true. Throw NotFoundError if not found ● **deleteMember(id: string): Promise<void>** - Delete member

	and user atomically. Throw NotFoundError if not found
exercise-service.ts	Exports the following functions: ● **createExercise(exerciseData: CreateExercise): Promise<Exercise>** - Validate input via schema, throw ConflictError if name duplicate. Delegate to repository ● **listExercises(page: number, searchQuery?: string, includeInactive?: boolean): Promise<Exercise[]>** - Fetch paginated exercises from repository ● **listActiveExercises(): Promise<Exercise[]>** - Fetch all active exercises (for dropdowns) ● **getExerciseById(id: string): Promise<Exercise>** - Fetch exercise, throw NotFoundError if not found ● **updateExercise(id: string, exerciseData: UpdateExercise): Promise<Exercise>** - Validate input via schema, throw ConflictError if name duplicate, throw NotFoundError if not found ● **archiveExercise(id: string): Promise<Exercise>** - Set isActive = false. Throw NotFoundError if not found ● **unarchiveExercise(id: string): Promise<Exercise>** - Set isActive = true. Throw NotFoundError if not found ● **deleteExercise(id: string): Promise<void>** - Delete exercise. Throw NotFoundError if not found
workout-session-service.ts	Exports the following functions: ● **createWorkoutSession(sessionData: CreateWorkoutSession, sessionExercises: CreateWorkoutSessionExercise[]): Promise<WorkoutSession>** - Validate all inputs via schemas. Throw SessionRequiresExercisesError if 0 exercises. Atomic call workout-session-repository.create(sessionData, sessionExercises) which creates session and all exercises in a single transaction (WSM-04). Throw NotFoundError if member/exercise not found ● **listWorkoutSessionsByMember(memberId: string, page: number): Promise<WorkoutSession[]>** - Throw NotFoundError if member is not found. Fetch from repository (includes exercises) ● **getSessionById(id: string): Promise<WorkoutSession>** - Fetch session with details. Throw NotFoundError if not found ● **updateWorkoutSession(id: string, sessionData: UpdateWorkoutSession): Promise<WorkoutSession>** - Validate input via schema. Throw NotFoundError if not found ● **addWorkoutSessionExercise(sessionId: string, exerciseData: CreateWorkoutSessionExercise):**

	Promise<WorkoutSessionExercise> - Validate input via schema. Call workout-session-exercise-repository.create(). Throw NotFoundError if session/exercise not found ● **removeWorkoutSessionExercise(sessionExerciseId: string): Promise<void>** - Check remaining exercise count first via workout-session-exercise-repository.countBySessionId(). Throw SessionRequiresExercisesError if last one. Call workout-session-exercise-repository.delete(). Throw NotFoundError if not found ● **updateSessionExercise(sessionExerciseId: string, exerciseData: UpdateWorkoutSessionExercise): Promise<WorkoutSessionExercise>** - Validate input via schema. Call workout-session-exercise-repository.update(). Throw NotFoundError if not found ● **deleteSession(id: string): Promise<void>** - Call workout-session-repository.delete(). This cascades to all exercises. Throw NotFoundError if not found
report-service.ts	Exports the following functions: ● **generateProgressReport(memberId: string, startDate: Date, endDate: Date): Promise<Report>** - Validate dates are valid ISO strings. Throw NotFoundError if member is not found. Validate endDate >= startDate. Call repository.getProgressReportData(). Return the computed Report.

Controller Layer

File	Responsibility
auth-controller.ts	Exports the following functions: ● **login(email: string, password: string): Promise<ActionResult<{ redirectUrl: string }>>** - Validate email/password via Zod schema. Call auth-service.login(). Create an iron-session cookie. Return { success: true, data: { redirectUrl } } or { success: false, message, errors }. Catch AuthorizationError -> form banner. Catch unexpected -> log and generic message ● **logout(): Promise<ActionResult<void>>** - Destroy session. Return { success: true }
member-controller.ts	Exports the following functions: ● **createMember(data: CreateMember): Promise<ActionResult<Member & { tempPassword: string }>>** - Validate via Zod schema. Check admin auth. Call member-service.createMember(). Return { success: true, data: { member, tempPassword } } or { success: false,

	message, errors }. Catch ConflictError (duplicate email) -> form banner. Catch unexpected -> log and generic message ● **updateMember(id: string, data: UpdateMember): Promise<ActionResult<Member>>** - Validate via Zod. Check admin auth. Call member-service.updateMember(). Return success or error. Catch NotFoundError, ConflictError ● **getMembers(page: number = 1, searchQuery?: string): Promise<ActionResult<Member[]>>** - Check admin auth. Call member-service.listMembers(page, searchQuery). Return paginated data or error ● **getMemberById(id: string): Promise<ActionResult<Member>>** - Check admin auth. Call member-service.getMemberById(). Return member or { success: false, message: 'Not found' } ● **suspendMember(id: string): Promise<ActionResult<Member>>** - Check admin auth. Call member-service.suspendMember(). Return updated member or error. Catch NotFoundError ● **unsuspendMember(id: string): Promise<ActionResult<Member>>** - Check admin auth. Call member-service.unsuspendMember(). Return updated member or error. Catch NotFoundError
exercise-controller.ts	Exports the following functions: ● **createExercise(data: CreateExercise): Promise<ActionResult<Exercise>>** - Validate via Zod. Check admin auth. Call exercise-service.createExercise(). Return { success: true, data: exercise } or { success: false, message, errors }. Catch ConflictError (duplicate name) -> form banner ● **updateExercise(id: string, data: UpdateExercise): Promise<ActionResult<Exercise>>** - Validate via Zod. Check admin auth. Call exercise-service.updateExercise(). Return success or error. Catch NotFoundError, ConflictError ● **getExercises(page: number = 1, searchQuery?: string, includeInactive: boolean = false): Promise<ActionResult<Exercise[]>>** - Call exercise-service.listExercises(page, searchQuery, includeInactive). Return paginated data (admin sees archive toggle option; members see active only) ● **getExerciseById(id: string): Promise<ActionResult<Exercise>>** - Call exercise-service.getExerciseById(). Return exercise or error ● **archiveExercise(id: string): Promise<ActionResult<Exercise>>** - Check admin auth. Call exercise-service.archiveExercise(). Return updated exercise or error. Catch NotFoundError

	• **unarchiveExercise(id: string): Promise<ActionResult<Exercise>>** - Check admin auth. Call exercise-service.unarchiveExercise(). Return updated exercise or error. Catch NotFoundError
workout-session-controller.ts	Exports the following functions: • **createWorkoutSession(sessionData: CreateWorkoutSession, sessionExercises: CreateWorkoutSessionExercise[]): Promise<ActionResult<WorkoutSession>>** - Validate all inputs via Zod schemas. Check admin auth. Throw SessionRequiresExercisesError if 0 exercises. Call workout-session-service.createWorkoutSession(). Creates session and all exercises atomically, in a single transaction (WSM-04). Return { success: true, data: session } or { success: false, message, errors }. Catch SessionRequiresExercisesError, NotFoundError (member/exercise) -> form banner • **updateWorkoutSession(id: string, data: UpdateWorkoutSession): Promise<ActionResult<WorkoutSession>>** - Validate via Zod. Check admin auth. Call workout-session-service.updateWorkoutSession(). Return updated session or error • **getSessionsForMember(memberId: string, page: number = 1): Promise<ActionResult<WorkoutSession[]>>** - Check admin auth. Validate memberId exists. Call workout-session-service.listWorkoutSessionsByMember(memberId, page). Return paginated sessions or error • **addSessionExercise(sessionId: string, exerciseData: CreateWorkoutSessionExercise): Promise<ActionResult<WorkoutSessionExercise>>** - Validate via Zod. Check admin auth. Call workout-session-service.addWorkoutSessionExercise(). Return exercise or error. Catch NotFoundError • **removeSessionExercise(sessionExerciseId: string): Promise<ActionResult<void>>** - Check admin auth. Call workout-session-service.removeWorkoutSessionExercise(). Return { success: true } or { success: false, message }. Catch SessionRequiresExercisesError (can't remove last), NotFoundError • **updateSessionExercise(sessionExerciseId: string, data: UpdateWorkoutSessionExercise): Promise<ActionResult<WorkoutSessionExercise>>** - Validate via Zod. Check admin auth. Call workout-session-service.updateSessionExercise(). Return updated exercise or error • **deleteWorkoutSession(id: string):**

	Promise<ActionResult<void>> - Check admin auth. Call workout-session-service.deleteSession(). Return { success: true } or { success: false, message }. Catch NotFoundError
report-controller.ts	Exports the following functions: ● **generateProgressReport(memberId: string, dateFrom: string, dateTo: string): Promise<ActionResult<Report>>** - Validate dates via Zod. Check auth: admin -> any member; member -> own only. Validate memberId exists. Call report-service.generateProgressReport(memberId, dateFrom, dateTo). Return { success: true, data: report } or { success: false, message }. Catch NotFoundError, AuthorizationError

Note: All controllers return a typed ActionResult.

Presentation Layer - Admin Routes

Route	Component	Responsibility
/admin/dashboard	/admin/dashboard/page.tsx	**Overview dashboard**. Displays the total members count, sessions this week count, quick links to create member/exercise/session. Fetches data via getMembers(), getSessionsForMember() server actions.
/admin/members	/admin/members/page.tsx	**Paginated, searchable member list**. Input: page, searchQuery. Calls getMembers() server action. Render member-list.tsx component. Links to detail, edit, suspend/unsuspend, delete.
/admin/members/new	/admin/members/new.page.tsx	**Member creation form**. Renders member-form.tsx component (create mode). Form action: createMember() server action. On success: show temp password in modal, then redirect to /admin/members. On error: show inline field errors and form banner.
/admin/members/[id]	/admin/members/[id]/page.tsx	**Member detail page**. Fetches via getMemberById() server action. Displays fullName, email, phone, dateOfBirth, membershipStart, isActive status. Show recent sessions. Links to edit member, view progress report, suspend/unsuspend member.
/admin/members/[id]/edit	/admin/members/[id]/edit/page.tsx	**Member edit form**. Fetches via getMemberById() server action. Pre-fill form with member data. Render member-form.tsx component (edit

		mode). Form action: updateMember() server action. On success: redirect to detail page. Show errors if a unique email constraint is violated.
/admin/exercises	/admin/exercises/page.tsx	**Exercise catalogue. Paginated, searchable table**. Input: page, searchQuery. Call getExercises() server action (includeInactive = true for admin). Render exercise-list.tsx component. Show archive toggle button per exercise. Links to edit, delete button.
/admin/exercises/new	/admin/exercises/new/page.tsx	**Exercise creation form**. Renders exercise-form.tsx component (create mode). Fields: name (8-64), description (0-1024), muscleGroup dropdown (CHEST, SHOULDERS, ARMS, BACK, CORE, LEGS), equipment dropdown (CABLE, DUMBBELL, BARBELL, MACHINE). Form action: createExercise() server action. On success: redirect to /admin/exercises. Shows an error for a duplicated name.
/admin/exercises/[id]/edit	/admin/exercises/[id]/edit/page.tsx	**Exercise edit form**. Fetches via getExerciseById() server action. Pre-fill form. Renders exercise-form.tsx component (edit mode). Form action: updateExercise() server action. On success: redirect to /admin/exercises.
/admin/sessions/new	/admin/sessions/new/page.tsx	**Log workout session form**. Renders session-log-form.tsx component. Fields: member dropdown (from getMembers()), date (ISO), duration (0-180 min), optional notes. Dynamic add/remove exercises rows: exercise dropdown (isActive = true only), sets (0-6), reps (0-30), weight (0.0-500.0). Validate min 1 exercise. Form action: createWorkoutSession() server action. Atomically creates a session and all exercises in a single transaction. On success: redirect to /admin/dashboard. Show errors if member/exercise not found, or validation fails.
/admin/sessions/[id]/edit	/admin/sessions/[id]/edit/page.tsx	**Edit session form**. Fetches sessions via getSessionsForMember() or similar. Shows session metadata (date, duration, notes) in edit form. Below: list existing exercises with individual edit/remove buttons. Each remove triggers removeSessionExercise() server action (validates min 1 remains). "Add exercise" button opens an inline form to addSessionExercise() server

		action. Session metadata update via updateWorkoutSession() server action.
/admin/members/[id]/report	/admin/members/[id]/report/page.tsx	**Progress report for a member**. Inputs: dateFrom, dateTo (ISO date picker). Member ID from URL. Call generateProgressReport(memberId, dateFrom, dateTo) server action. Render progress-report.tsx component. Display: totalSessions, totalVolume, averageSessionDuration, per-exercise breakdown table, chronological session table with exercises.

Presentation Layer - Member Routes

Route	Component	Responsibility
/member/dashboard	/member/dashboard/page.tsx	**Welcome dashboard**. Displays member fullName, membershipStart date, quick stats (total sessions all time, total volume all time). Links to view sessions, view report, change password.
/member/profile	/member/profile/page.tsx	**Read-only personal profile**. Displays fullName, email, phone, dateOfBirth, membershipStart, isActive. "Change password" button -> link to change-password page. Data from session context (no server action needed).
/member/sessions	/member/sessions/page.tsx	**Paginated list of own sessions (read-only)**. Input: page. Fetches via getSessionsForMember(currentMemberId, page) server action. Render session-list.tsx component (read-only mode). Sorted descending by date. Links to session detail.
/member/sessions/[id]	/member/sessions/[id]/page.tsx	**Session detail (read-only)**. Fetches via getSessionsForMember() or similar. Display: date, duration, notes, exercises table (exerciseName, sets, reps, weight per row).
/member/report	/member/report/page.tsx	**Progress report (own data only)**. Inputs: dateFrom, dateTo (ISO date picker). Call generateProgressReport(currentMemberId, dateFrom, dateTo) server action (backend checks auth: member can only view own). Render progress-report.tsx component.

Shared Components

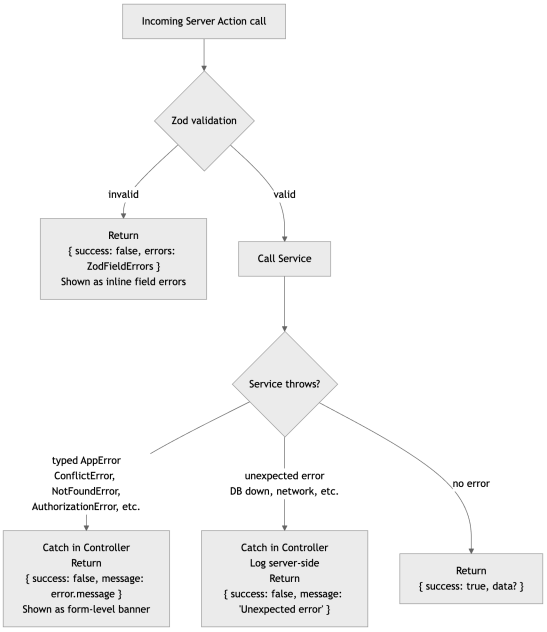
Component	Responsibility
auth/login-form.tsx	**Client component**. Form: email and password inputs, login button. Form action: login() server action. On success: use Next.js useRouter to redirect to returned redirectUrl (admin dashboard or member dashboard). On error: show field errors (inline) and form banner with error message
members/member-form.tsx	**Client component**. Controlled form with Zod client-side validation (createMemberSchema or updateMemberSchema). Mode: 'create' or 'edit'. Create mode: locked email input (will be generated). Edit mode: pre-filled inputs, email unique check against DB. Fields: fullName (8-64), email, phone, dateOfBirth (date picker, ISO), membershipStart (date picker, ISO). Form action: createMember() or updateMember() server action. Show field errors inline, form-level error banner. On create success: show modal with temp password (copy button)
members/member-list.tsx	**Client component**. Renders paginated table. Props: members[],, currentPage, searchQuery, onPageChange, onSearch callbacks. Columns: fullName, email, phone, isActive badge, actions (detail, edit, suspend/unsuspend). Suspend/unsuspend button triggers suspendMember()/unsuspendMember() server action and refetch
exercises/exercise-form.tsx	**Client component**. Controlled form with Zod validation (createExerciseSchema or updateExerciseSchema). Mode: 'create' or 'edit'. Fields: name (8-64), description (0-1024 textarea), muscleGroup dropdown (CHEST, SHOULDERS, ARMS, BACK, CORE, LEGS), equipment dropdown (CABLE, DUMBBELL, BARBELL, MACHINE). Form action: createExercise() or updateExercise() server action. Show field errors inline, form banner. On success: redirect
exercises/exercise-list.tsx	**Client component**. Renders paginated table. Props: exercises[], totalCount, currentPage, searchQuery, isAdmin boolean. Columns: name, muscleGroup badge, equipment badge, isActive (archived status if admin). Admin sees: archive/unarchive toggle button, edit link, delete button. Member sees: read-only view (no toggles/buttons). Pagination controls, search input
sessions/session-log-form.tsx	**Client component**. Multi-row exercise form. Props: availableMembers[], availableExercises[]. Dynamic add/remove exercise rows. Member dropdown (single select). Date input (ISO). Duration input (0-180). Notes textarea. Each exercise row: exercise dropdown (isActive = true only), sets spinner (0-6), reps spinner (0-30), weight input (0.0-500.0 decimal). Add row button, remove row button (validates min 1). Form action: createWorkoutSession() server action. On success: show success message and redirect. On error: show field errors and banner
sessions/session-list.tsx	**Client component**. Renders chronological session table (sorted descending). Props: sessions[], totalCount, currentPage, isReadOnly

	boolean. Columns: date, duration, exercise count / exercise names. Pagination controls. Links to detail. If not read-only (admin): delete button triggers deleteWorkoutSession() server action and refetch
reports/progress-report.tsx	**Client component**. Props: report (Report type). Render summary section: totalSessions, totalVolume, averageSessionDuration. Per-exercise breakdown table: exercise name, totalSets, maxWeightKg. Chronological session table: date, duration, exercises (nested rows). Date range inputs (dateFrom, dateTo) above report. "Apply filter" button triggers generateProgressReport() server action and re-render with new data
ui/	**Reusable primitives**. Button, Input, Select (dropdown), Badge (isActive status), Table (pagination-aware), Modal, Textarea, DatePicker (ISO input), Spinner (for numeric inputs 0-6, 0-30, etc). All unstyled-forward compatible with Tailwind CSS

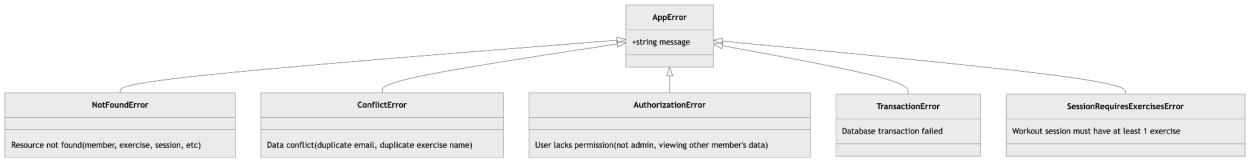
Error Handling Strategy

All errors are categorised into three tiers. No stack trace is ever sent to the client.

Error Flow



Custom Error Hierarchy



Error Tier Summary

Tier	Source	Caught By	View	Logging
Validation	Zod schema failure	Controller Server Actions	Inline field errors	-
Business Rule	Service throws typed AppError	Controller Server Actions	Form-level error banner	-
Infrastructure	Unexpected database / network error	Controller Server Actions	Generic error message	console.error

Design Patterns Applied

Pattern	Application	Description
Repository	/lib/repository	Isolates all database interaction related code. Services , controllers and the view have no ORM knowledge.
Service Layer (Facade)	/lib/service	Encapsulates business logic and multi-step operations. Controllers stay thin (validate -> call -> return).
Singleton	/lib/database.ts	Single shared Prisma client instance to avoid connection pool exhaustion.
Command	/lib/controller	Each Server Action is a discrete named command (e.g. createWorkoutSession), independently testable.
Strategy	/lib/service/report-service.ts	Report aggregation is isolated. The computation strategy can be extended without touching any other layer.

Accommodating Future Changes

Possible change	How the architecture handles it
Export report as PDF	Add a pdf-report-adapter.ts that wraps the report-service.ts generated report, no service changes.
Add charts to the report	Report already contains per-session per-exercise arrays. Only the progress-report.tsx component needs updating.
Swap PostgreSQL for another database	Prisma easily accommodates this change, with the change of just a few lines, and running one command.
Add REST APIs	Add app/api/ route handlers that call the same services.

Multi-gym / multi-tenant	Add a gymId FK to Member, Exercise, and WorkoutSession. Enforce in repository queries.